

# Projektuppgift

*Frontend-baserad webbutveckling*

**DailyBread**

Kristen böneapp

**Weronika Sleboda**



**Mittuniversitetet**

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. Campus Sundsvall Holmgatan 10, SE-851 70 Sundsvall.

Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.

Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

**MITTUNIVERSITETET**

**Avdelningen för informationssystem och -teknologi**

**Författare:** Weronika Sleboda [wesl2400@student.miun.se](mailto:wesl2400@student.miun.se)

**Utbildningsprogram:** Webbutveckling, 120 hp

**Huvudområde:** Datateknik

**Termin, år:** VT, 2026

## Sammanfattning

DailyBread är en böneapp avsedd för kristna inom det messianska trossamfundet. Appen väljer ut en relevant bön ur Psalmboken och Nya Testamentet baserat på väderprognosen, helgdagen shabbat och användarens region. Applikationen är en frontend-webbapplikation och en mashup som hämtar data från flera API:er med hjälp av JavaScript och AJAX och kombinerar dessa i en gemensam funktionalitet. Utvecklingsprocessen resulterar i en böneapp som tar hänsyn till användarens regionala kontext, trossamfund och eventuella väderrelaterade sinnestillstånd.

**Nyckelord:** JavaScript, AJAX, API, Frontend, Mashup

# Innehållsförteckning

|                                              |             |
|----------------------------------------------|-------------|
| <b>Sammanfattning.....</b>                   | <b>iii</b>  |
| <b>1    Introduktion.....</b>                | <b>v</b>    |
| 1.1    Problemformulering.....               | v           |
| 1.2    Problemlösning.....                   | v           |
| <b>2    Teori.....</b>                       | <b>vi</b>   |
| 2.1    API, REST och HTTP-metoder.....       | vi          |
| 2.2    Asynkront JavaScript.....             | vi          |
| 2.3    Promise, await och Promise.all()..... | vii         |
| 2.4    Ajax.....                             | vii         |
| 2.5    Single Page Application.....          | viii        |
| 2.6    MVC.....                              | viii        |
| 2.7    CSS-preprocessorer och Sass.....      | viii        |
| 2.8    CSS-animationer och keyframes.....    | viii        |
| 2.9    Mashup.....                           | viii        |
| 2.10    JSON.....                            | ix          |
| 2.11    Automatisering.....                  | ix          |
| 2.12    Vite.....                            | ix          |
| 2.13    Github Actions.....                  | ix          |
| <b>3    Metod.....</b>                       | <b>x</b>    |
| 3.1    Arbetsflöde.....                      | x           |
| 3.2    Bibelmotor.....                       | xi          |
| 3.3    Verktyg.....                          | xii         |
| 3.4    Testning och publicering.....         | xii         |
| <b>4    Konstruktion.....</b>                | <b>xiii</b> |
| 4.1    Funktioner och sidostruktur.....      | xiii        |
| 4.2    Nätverkslogik.....                    | xiii        |
| 4.3    Bibelmotorns logik.....               | xiv         |
| 4.4    UI-logik.....                         | xiv         |
| 4.5    Stilisering med Sass.....             | xv          |
| 4.6    Animationer.....                      | xv          |
| 4.7    Arkitektur.....                       | xv          |
| 4.8    Grafisk profil.....                   | xvi         |
| 4.9    Automatisering och publicering.....   | xix         |
| 4.10    Testning.....                        | xix         |
| <b>5    Resultat.....</b>                    | <b>xx</b>   |
| <b>6    Slutsatser.....</b>                  | <b>xxi</b>  |
| <b>Källförteckning.....</b>                  | <b>xxii</b> |

# 1 Introduktion

## 1.1 Problemformulering

DailyBread är en böneapp som riktar sig till kristna inom det messianska trossamfundet. Detta trossamfund är en del av den globala evangelikala kyrkan som praktiserar kristendomen med hänsyn till Bibelns judiska kontext. Eftersom trossamfundet består till stor del av etniska judar och personer av judisk härkomst samt behåller många judiska riter, kallas ibland rörelsen för 'messiansk judendom'. Denna benämning är dock inte alltid korrekt med tanke på att även icke-judar ingår i rörelsen [1].

Böneplatser för de messianska troende kallas både för synagogor och kyrkor och enligt uppgifterna från den internationella messianska organisationen IAMCS finns de flesta i USA, Ukraina och Israel [2]. Samfundet är alltså en minoritetskyrka och är ofta underrepresenterad både i samhället och online. Även trots att det redan existerar många kristna webbplatser och appar, exkluderar dessa ofta messianska utövare utifrån teologin och deras behov.

Det finns fortfarande få kristna webbplatser som ger möjligheten att spåra tider för helgdagen shabbat för de som firar den. Tanken bakom appen DailyBread är därför att adressera de behov som idag inte är tillgodosedda av andra kristna appar och webbplatser.

## 1.2 Problemlösning

Appen ska vara politiskt oberoende och inte exkludera användare baserat på härkomst eller region. Tvärtom, appen ska ta hänsyn till användarens region, både utifrån regionens rådande förhållanden samt dess väderfaktorer. En kristen bosatt i ett land med hög religiös förföljelse behöver ha fler förföljelserelaterade böner, medan en som är bosatt i västeuropa behöver fler böner som stärker tron. Det är nämligen lätt att tappa tron i sekulära länder där kyrkor stängs och byggs om till nattklubbar, hotell och klädbutiker [3]. Troende från Israel får däremot en egen dedikerad kategori av böner som anknyter till den judiska identiteten.

I regioner som Skandinavien med ett konstant dåligt väder, förekommer väderorsakad depression oftare [4], till skillnad från till exempel Spanien, där människor i större grad exponeras för solen. Solen bidrar till produktionen av lyckohormonet 'serotonin' som gör människor glada [5]. DailyBread-appen är medveten om det och anpassar böner efter väderförhållanden. Bönerna blir upplyftande eller tröstande beroende på om väder är bra eller dåligt. Appen tar också hänsyn till shabbattider och genererar shabbatrelevanta bibelverser under shabbatdagen.

## 2 Teori

### 2.1 API, REST och HTTP-metoder

API står för Application Programming Interface och är ett gränssnitt som erbjuder olika typer av återanvändbar funktionalitet [6]. Detta sker via bestämda regler [7] som påminner om ett kontrakt [6]. Program som använder API:er kallas för klienter och API:er fungerar i denna kontext som tjänster som klienterna tar del av. Inom webbutveckling syftar API specifikt på en uppsättning metoder, egenskaper, event med mera som kan integreras i andra applikationer. Ett exempel på ett API är Geolocation API som kan inkorporeras i webbappar för att hämta användares platsdata [8].

REST är förkortning för Representational State Transfer och är en arkitektur för API:er. Detta innebär att REST bestämmer hur ett API ska utformas och fungera. Arkitekturen har utformats i syfte att möjliggöra kommunikationen mellan olika plattformar och göra denna kommunikation skalbar, modifierbar och effektiv. API:er som utvecklas i enlighet med REST-arkitekturen kallas för REST-API:er och i kontexten av webbapplikationer, kallas de för RESTful webb-API:er [8].

Det som kännetecknar REST-arkitekturen är att den är tillståndslös, vilket innebär att varje förfråga som skickas till ett API är oberoende av andra förfrågningar. Förfrågningar använder sig av den så kallade 'uniform resource identifier' [7] (förkortning: URI) för att identifiera resurser som efterfrågas. Klienter erhåller tillräcklig information från tjänsten för att genomföra en förfråga samt hämta, modifiera eller radera en resurs. Det sista sker med hjälp av HTTP-metoder: GET, POST, PUT och DELETE. GET hämtar en resurs och DELETE raderar en resurs. Både POST och PUT skapar ny resurs men POST duplicerar en resurs om samma förfråga skickas flera gånger, medan PUT uppdaterar en resurs om den redan existerar och om samma förfråga skickas på nytt [8].

### 2.2 Asynkront JavaScript

JavaScript är enkeltrådat, vilket innebär att det exekverar instruktioner i en sekvens åt gången [9], till skillnad från till exempel Java som är kapabel av att exekvera instruktioner i flera trådar; det vill säga i flera oberoende sekvenser samtidigt [10]. JavaScript kör instruktioner en efter en i nedstigande led i webbläsarens huvudtråd. Huvudtråden sköter som standard både exekveringen av JavaScript och återgivningen av det grafiska användargränssnittet. Detta leder ibland till att grafiska komponenter inte svarar i väntan på att Javascript koden slutför alla sina instruktioner [11].

Program där instruktioner kör i en bestämd ordning, en efter en och väntar på varandra, kallas för synkrona program. I asynkrona program sker motsatsen; instruktioner har en bestämd ordning men den ändras under exekveringen via så kallade 'callbacks'. En 'callback' är en funktion som skickas till en annan funktion, vilket gör det möjligt för instruktioner att kommunicera och hoppa mellan varandra under exekveringen. På det här sättet, kan flera instruktioner köra samtidigt, dock inte på samma sätt som i en matbutik med flera kassor och flera arbetare utan som i en matbutik med en kassa där flera arbetare avlöser varandra. DOM händelser är ett exempel på asynkron exekvering. En händelse kör inte direkt utan enbart när det är dags för den att köra [9].

## 2.3 Promise, await och Promise.all()

Callback-funktioner är ett sätt att uppnå asynkronitet i JavaScript men tekniken är föråldrad och leder till en djupt nästlad och svårläst kod [9]. En annan teknik är att använda Promise-objekt och -funktioner. Ett Promise-objekt har tre olika tillstånd: 'pending', 'fulfilled' och 'rejected'. När ett promise-objekt inte ännu har slutförts befinner det sig i 'pending' tillståndet, när det har lyckats är tillståndet 'fulfilled' och när det har misslyckats är tillståndet 'rejected' [12]. Promise-objekt och funktioner exekverar asynkront på samma sätt som callback-funktioner men de gör detta på ett mer optimalt sätt [9]. För att initiera ett Promise-objekt måste två callback-funktioner specificeras: en som kallas när exekveringen slutförs utan problem och en som kallas vid fel. Därav heter den första 'resolve' och den andra 'reject' [12].

Både nyckleordet 'await' [13] och metoden then() kallas för att invänta ett Promise-objekt och hämta dess resultat [12], med den skillnaden att nyckelordet 'await' kortar ner koden och gör den mer läsbar [13]. När flera Promise-objekt beror på varandras resultat bör dem inväntas en efter en med hjälp av nyckelordet 'await'. I andra fall är det mest effektivt att definiera dessa i en lista inom Promise.all-funktionen. Promise.all-funktionen avfyrrar flera oberoende Promise-objekt samtidigt, vilket gör att resultaten inkommer snabbare [14].

## 2.4 Ajax

Ajax är en förkortning för asynkront JavaScript och XML och är ett samlat namn på webbutvecklingstekniker som gör det möjligt för en app att uppdatera delar av sitt innehåll utan att det behöver laddas om i sin helhet. Detta sker via asynkrona HTTP-förfrågningar till servern. Användning av Fetch API är ett sätt att uppnå detta [15]. Metoden fetch() skickar en asynkron HTTP förfråga för att hämta resurser från servern och returnerar ett Promise-objekt med resultatet [16].

## 2.5 Single Page Application

En single-page-applikation är en applikation som består av ett enda HTML-dokument som uppdateras dynamiskt via JavaScript. Den största fördelen med single-page-applikationer är deras höga prestanda och responsivitet, eftersom det enstaka HTML-dokumentet de bygger på, laddas bara en gång. Dokumentets innehåll uppdateras senare i delar, till skillnad från de traditionella flersidiga applikationerna där innehållet uppdateras genom att en ny undersida laddas. Single-page-applikationer har dock också sina nackdelar och den största är att de är svåra att sökmotoroptimera [17]. Det mesta av deras innehåll är nämligen inte ännu skapat vid tidpunkten för sökmotorindexeringen [18].

## 2.6 MVC

MVC är förkortning för Model View Controller och är ett namn på en arkitektur inom systemutveckling. Arkitekturen delar in koden i ett system i tre delar: modeller, vyer och kontrollanter. Modeller sköter systemets logik, vyer tar hand av det grafiska gränssnittet, medan kontrollanter koordinerar samarbetet mellan modellerna och vyerna så att dessa är separerade från varandra [19]. Syftet med arkitekturen är att minska beroenden mellan olika systemdelar; särskilt mellan logiken och det grafiska gränssnittet, så att systemet är lättare att utöka och underhålla [20].

## 2.7 CSS-preprocessorer och Sass

En CSS-preprocessor är ett verktyg som effektiviserar arbetet med CSS genom att generera CSS filer via sin egen syntax. Denna syntax erbjuder ofta funktionalitet som ursprungligen inte finns i CSS. Ett exempel på en CSS-preprocessor är Sass [21]. Sass är ett CSS-kompatibelt stilmallsspråk som översätts till CSS via kompilering. Preprocessorn utökar CSS med variabler, nästlade regler, funktioner med mera som gör det lättare att organisera [22], modifiera samt underhålla stora CSS-kodbaser [22].

## 2.8 CSS-animationer och keyframes

CSS-animationer är en modul inom CSS som möjliggör animationer av HTML-element via så kallade @keyframes. @keyframes är en uppsättning av CSS-regler som definierar hur CSS-egenskaper ska förändras vid olika punkter i en animationssekvens [24] under ett visst tidsintervall [23].

## 2.9 Mashup

En mashup är en webbutvecklingsteknik, där resurser och funktionalitet från flera olika källor - vanligen offentliga och kostnadsfria API:er – kombineras i syfte att skapa en ny tjänst [25].



## 2.10 JSON

JSON är en förkortning för JavaScript Object Notation och är ett dataöverföringsformat som består av samlingar av nyckelord och deras motsvarande värden samt av ordnade listor. Formatet är lätt både för människor att förstå och för datorer att bearbeta och generera. Det har också bra stöd bland de flesta programmeringsspråk [38] och används ofta för att representera serverrespons i RESTful API:er [7].

## 2.11 Automatisering

Inom systemutveckling syftar 'automatisering' på användning av program och skript för att automatiskt utföra specifika uppgifter som annars måste utföras manuellt av människor med hjälp av datorer. Uppgifterna kan bland annat omfatta testning, publicering och övervakning av mjukvara [40].

## 2.12 Vite

Vite är ett byggverktyg som på flera sätt förenklar utveckling av frontend-applikationer. Verktyget skapar bland annat en färdig projektstruktur, erbjuder en testserver samt optimerar projektets filer och buntar dem inför publicering [41].

## 2.13 Github Actions

Github Actions är en plattform som automatiserar byggprocesser, testning samt publicering av mjukvaruprojekt med hjälp av så kallade arbetsflöden. Arbetsflöden skrivs i [42] dataseriatiseringsspråket YAML [43] och består av en uppsättning av uppgifter som automatiskt ska utföras varje gång ändringar skickas till kodbasen på Github-servern [42].

## 3 Metod

### 3.1 Arbetsflöde

Arbetsflödet delas in i fyra faser: implementation av logik, utveckling av ett grafiskt gränssnitt, integration av logiken och gränssnittet samt stilisering av gränssnittet med hjälp av Sass. Det finns flera fördelar med en sådan arbetsuppdelning. För det första är applikationen lättare att testa. Det räcker att skriva några rader kod i `main.js`-filen och logga resultatet i terminalen för att omedelbart avgöra om ett API lämpar sig för det avsedda ändamålet. Potentiella problem och misstag kan upptäckas tidigt och åtgärdas. Endpoints kan snabbt utvärderas utifrån hur väl de uppfyller applikationens krav. Vid den här tidpunkten går applikationens koncept fortfarande att omarbetas och till och med ersättas av ett bättre alternativ. Det kan hända att ett koncept lyckas i skisserna men misslyckas senare i praktiken. Denna arbetsmetod eliminerar risken att ett helt gränssnitt måste skrotas när det visar sig att det bygger på en idé som inte fungerar.

Anledningen till att utvecklingen och stiliseringen av vyerna sker i två separata faser är att skapa ett flexibelt gränssnitt. På detta sätt kan vyerna applicera en valfri stilmall utan att gränssnittets struktur förstörs. UI utvecklas i form av återanvändbara och stilmalloberoende komponenter med interaktiva delar såsom knappar och inmatningsfält. Komponenterna fungerar som byggstenar för vyer. Denna arkitektur inspireras av ramverk som Android API [26], Java Swing [27] och C++ WxWidgets [28], där UI-komponenterna ofta har en inbyggd interaktiv funktionalitet.

Att organisera arbetet i faser med tydliga fokuspunkter och väldefinierade uppgifter minskar distraktioner och förbättrar testningen. Varje del av applikationen testas i isolering och godkänns inte förrän den är felfri. Att åtgärda UI- och logikrelaterade problem samtidigt leder till att flera fel missas eller sprids över hela systemet. Det kan hända att ett fel framstår som ett UI-fel men orsakas av logik på en djupare systemnivå, eller vice versa. Det kan uppstå en situation där en väl fungerande logik rättas på grund av ett felaktigt diagnostiserat UI-fel. Som ett resultat slutar både logiken och gränssnittet att fungera, vilket leder till att ett fel utvecklas till två.

Separationen av de olika delsystem ger också friheten att implementera den mest lämpliga arkitekturen och inte den enda möjliga. När logik och gränssnitt utvecklas parallellt ökar antalet beroenden mellan dessa och varje extra beroende minskar urvalet av tillgängliga arkitekturlösningar. Situationen kan liknas vid ett scenario där en råtta flyr från en katt och anpassar sina steg efter kattens rörelse och inte efter vad som är smartast att göra i den givna stunden. I slutändan tvingas råtten i ett hörn och står kvar med det enda möjliga alternativet – att låta sig bli uppäten.

Många gånger hamnar utvecklare i en liknande situation, där de måste acceptera en dålig arkitektur eftersom ingen annan är möjlig. Systemet är så tätt sammankopplat att den dåliga arkitekturen måste accepteras eller så måste hela systemet skrivas om från grunden.

Ett sådant arbetssätt ger också mer frihet när det kommer till design av Figma-prototyper. Eftersom både logiken och gränssnittet redan är implementerade, är antalet komponenter förutbestämda och kan arrangeras fritt. Layouten kan anta vilken form som helst, förutsatt att komponenternas innehåll respekteras. Komponenterna behöver inte heller utvärderas ur ett tekniskt perspektiv med tanke på att de redan har testats. Det enda de saknar är en grafisk profil, vilket gör att flera Figma-prototyper kan skapas parallellt och den bästa kan väljas.

## 3.2 Bibelmotor

Till skillnad från de redan existerande slumpmässiga bibelversgeneratorer, väljs slumpmässiga böner och verser i DailyBread-appen utifrån ett poängsystem. Poängen fördelas beroende på platsdata, väderprognos och shabbattider, där olika regioner, temperaturer och väderkoder producerar olika utfall. Data hämtas från API:er i realtid, vilket kan dröja beroende på nätverkshastigheten och antalet simultana HTTP-förfrågningar. Av den anledningen sker datahämtningen asynkront. På detta sätt säkerställs att responsen från API:er kommer i god tid samt att UI-komponenterna inte blockeras under processen.

Bibelverser delas i sex kategorier enligt tema: lovord, styrka, prov, förföljelse, Israel och helgdagen shabbat. Sista kategorin 'shabbat' är dock enbart tillgänglig under shabbatdagen. Kategorin 'lovord' utlöses av soliga väderkoder och temperaturer över 0 celsiusgrader. Undantaget är temperaturer över 26 celsiusgrader som enligt National Weather Service klassificeras som höga [30] – dessa utlöser kategorin 'prov', med tanke på att höga temperaturer kan bidra till psykiska besvär såsom nedsatt motivation, sömnlöshet, irritation och ångest [31]. Väderkoder för oväder tilldelas däremot kategorin 'styrka'. Samma kategori tilldelas också temperaturer som är lika med eller lägre än 0 celsiusgrader som ofta stärker känslan av ensamhet [32] och tyder på vinterperioden [33]. Under vintern förekommer depression nämligen oftare än under de övriga säsongerna [34].

Regioner delas utifrån den kulturella samhörigheten i två huvudregioner: väst och öst. Den västra regionen domineras av den västerländska kulturen och inkluderar kontinenterna: Nord- och Sydamerika, Europa och Australien, medan Asien och Afrika som förknippas med den österländska kulturen ingår i den östra regionen. Bibelkategorin för den västra regionen är 'prov'. Trots sin stora kristna befolkning [35] är regionens religionsutövande i reträtt på grund av sekularisering [36].

Den östra regionen tilldelas däremot kategorin 'förföljelse'. Även om södra Afrika och den asiatiska delen av Ryssland är i hög grad befolkade av kristna [35], visar en rapport från organisationen Global Christian Relief att regionen präglas av svår förföljelse gentemot den kristna folkgruppen [37]. Slutligen, får Israel sin egen 'Israel'-kategori. Denna kategori är en referens till en samling bibelverser som kristna från den regionen kan relatera till. Verserna nämner bland annat det judiska folket och historiska bibliska platser som ingår i dagens Israel.

För att undvika deterministiska utfall läggs till en slumpmässigt vald kategori av fem (borträknat shabbat-kategorin) i det slutgiltiga resultatet, så att användaren fortfarande får en chans att få en bibelvers ur de andra kategorierna. I systemet heter detta 'gudomlig faktor' och syftar just på slump. Mekanismen finns till för att uppväcka hos användaren en känsla av att denna kategorin har valts av Gud själv, vilket förbättrar användarupplevelsen hos den specifika målgruppen. Grundtanken bakom bibelmotorn är att anpassa upplevelsen efter en konkret användare, så att hen får en känsla av att appen riktar sig till hen personligen och inte till en oidentifierad, grå folkmassa.

### 3.3 Verktyg

Optimeringen och buntningen av filer hanteras med hjälp av byggverktyget Vite och publiceringen av projektet automatiseras via Github Actions. Applikationen testas både på en dator och på en Android-mobil (Motorola G24 Power) och i följande webbläsare: Firefox, Chrome (både dator- och Android-versionen), Microsoft Edge och Opera. Applikationens logik analyseras med hjälp av Firefox-verktyg och dess prestanda, sökmotoroptimering samt tillgänglighet övervakas via Chrome-tillägget Lighthouse. Därutöver utvecklas design-prototypen och logotypen i Figma.

### 3.4 Testning och publicering

Testning utförs framför allt via loggning i webbläsarkonsolen och omfattar testning av enstaka funktioner och separata features. Som det redan har nämnts under rubriken 'Arbetsflöde' testas affärslogiken och UI-logiken separat. UI-logiken testas på dator och mobil genom interaktion. Den färdiga applikationen publiceras på Github Pages och därefter används och felhanteras dagligen under perioden då rapport skrivs.

## 4 Konstruktion

### 4.1 Funktioner och sidostruktur

DailyBread-appens huvudfunktionalitet är att vissa relevanta visdomsord och böner baserat på rekommendationer från en inbyggd bibelmotor. Det finns också en sidofunktionalitet som visar shabbattider för en vald plats samt räknar antalet dagar kvar till shabbatdagen. Applikationen består endast av en HTML-sida med två kortvyer som dynamiskt byts ut mot varandra. Den första kortvyn tar emot platsdata och innehåller en kort introducerande information om hur DailyBread-appen fungerar. Även integritetspolicyn inkluderas i denna vy.

Den andra kortvyn visar visdomsord och böner. Vyerna delar samma header-element med nyckelord, en huvudbild och en sidfot som visar shabbattider för användarens plats. Sidfoten uppdateras också dynamiskt när knappen 'Pick My Prayer' trycks. Knappen är inkluderad i den första kortvyn som ansvarar för platsinmatning. En knapp som heter 'Start Over' inkluderas i den andra vyn för visdomsord och böner och tar användaren tillbaka till första vyn.

### 4.2 Nätverkslogik

API:er som används i applikationen är RESTful API:er som returnerar data i JSON-format. Dessa API:er inkluderar: OpenStreetMap för platsdata, OpenMeto för väderprognoser, BibleApi för bibelverser och Hebcal för shabbattider. AJAX-tekniker tillämpas via metoden `fetch()`, som skickar asynkrona förfrågningar till servrar och hämtar data med hjälp av HTTP-metoden GET. Eftersom huvudfunktionaliteten i DailyBread-appen är en mashup, beror flera API:er på varandras responser. Platsdata hämtas först och matas därefter dynamiskt in i HTTP-förfrågningar som skickas till väder- och shabbat-API:erna.

Av den anledningen inväntas först responsen från plats-API:t och därefter de sistnämnda API:erna. Detta sker med hjälp av nyckelordet `'await'` som föredras här framför Promise-metoden `then()` med motiveringen att det gör koden mer kortfattad och lättare att förstå. Förfrågningar till väder- och shabbat-API:erna exekveras parallellt i metoden `Promise.all()`, därför att dessa är oberoende av varandra och kan optimeras på detta sätt för bästa prestanda. Även förfrågningar till bibel-API:t definieras i metoden `Promise.all()` för att förkorta responstiden. Dessa är inte heller beroende av varandra och kan således utföras samtidigt.

Alla förfrågningar görs via hjälpfunktioner och dedikerade klasser. Hjälpfunktionerna abstraherar logiken bakom metoden `fetch()` och gör HTTP-förfrågningar smidigare och enklare att arbeta med. Klasserna delar in applikationens huvudfunktionalitet i delfunktioner, så kallade 'features' som ansvarar för platsdata, väderdata, shabbatdata, bibeldata och bibelmotorn. Dessa inkapslas i klasserna i syfte att effektivisera datahanteringen och testningen samt göra koden

tydligare. Det finns också modellklasser för respektive funktion som lagrar resultat från API:ernas JSON-responser i klassfält, vilket gör det möjligt att visa resultaten i HTML-element via attributet 'textContent'.

### 4.3 Bibelmotorns logik

Logiken för bibelmotorn finns inpackad i en dedikerad klass som heter CalScore. Denna klass tar emot plats-, väder- och shabbatdata i konstruktören och baserat på dessa beräknas en kategori ur vilken bibelverser ska hämtas. Resultatet av beräkningen är en array, alltså en lista. Listan består av enum-värden som representerar bibelkategorier. Bibelkategorierna läggs in i resultatlistan beroende på väderkoder, temperaturer, region, land och shabbattider. Det finns också en slumpmässig kategori av fem som läggs in i listan, den så kallade 'gudomliga faktorn' som nämns tidigare i rapporten under rubriken 'Bibelmotor'. Resultatlistan är således en lista av möjliga utfall och av denna lista väljs en slumpmässig kategori. Denna slumpmässiga kategorin är det slutgiltiga resultatet för bibelmotorns beräkning.

Det finns två datasamlingar av typen Map i applikationen som lagrar listor av bibelreferenser. Den konstanta Map-strukturen 'PSALMS' lagrar referenser till verser ur Psalmboken medan den konstanta Map-strukturen 'WISDOMS' lagrar referenser till verser ur Nya Testamentet. Referenserna är indelade i listor enligt bibelkategorier. När CalScore-klassen väljer en kategori, används kategorin sedan som nyckel för att hämta bibelreferenslista ur både Map-strukturen 'PSALMS' och Map-strukturen 'WISDOMS'. En slumpmässig referens väljs därefter ur respektive Map-struktur enligt den valda kategorin.

Kategorin för shabbat lossas upp endast när shabbatfunktionens shabbaträknare nollställs. Vanligen räknas tiden för helgdagen shabbat från solnedgången [39], men i applikationen räknas den från den dagen den infaller. Shabbaträknaren är också en extra funktion vid sidan av applikationens huvudfunktionalitet och den finns alltid synlig i sidfoten på startsidan.

### 4.4 UI-logik

Ett flertal hjälpfunktioner skapas i syfte att abstrahera och förenkla DOM-hantering och används senare för att utforma interaktiva UI-komponenter. Logiken för UI-komponenterna inpackas i klasser med suffixet 'Card' och återanvänds därefter för att arrangera kompletta vyer efter deras respektive huvudansvar. De färdiga vyerna heter: LocView, PrayView och ShabView, där LocView ansvarar främst för inmatning och visning av platsdata, Prayview för visning av böner och visdomsord och ShabView för visning av shabbattiderna och shabbaträknaren. Det finns också en vy som heter AppView. Den håller referensen till det HTML-element som LocView och PrayView kopplas till, samt referensen till sidfoten som ShabView kopplas till. Utöver detta instansierar vyn ett objekt av klassen App. Klassen App fungerar som en abstraktion över logiken som dynamiskt byter och uppdaterar vyerna med hjälp av callback-funktioner och enum-värden.

## 4.5 Stilisering med Sass

Sass-funktioner som används för att stilisera applikationens HTML-element är: ärvning, nästling, mixins, variabler och partials. Sass-koden delas in i följande moduler: bas, komponenter och inställningar. Bas-modulen innehåller flera partials med återanvändbara variabler för färger, mått och typsnitt samt layoutrelaterade mixins och basklasser för ärvning. Också keyframes för animationer ingår i denna modul. Som namnet antyder, innehåller komponent-modulen partials med regler för hela komponenter såsom knappar, logotyp, förloppsindikator, listor, kort-vyer med mera. Komponenterna och enstaka regler importeras slutligen i en ensamstående fil med namnet 'root'. En css-fil med samma namn länkas i index.html-filen där alla vyer visas.

## 4.6 Animationer

När knappen 'Pick My Prayer' trycks i kortvyn för platsdata, visas en förloppsindikator som är synkroniserad med flera asynkrona metoder som hämtar data från API:erna. När förloppsindikatorn når 100% försvinner platsdatavyn och vyn för visdomsord och böner tonas in med hjälp av en CSS-animation som skapas med hjälp av @keyframes. Båda animationerna finns till för att förbättra användarupplevelsen. Eftersom HTTP-förfrågningar ibland kan dröja är det viktigt att informera användaren att applikationen fortfarande arbetar och förbereder responsen. Den subtila intoningen när en ny vy visas, bidrar dessutom till att övergången mellan vyerna inte upplevs som hackig och abrupt.

## 4.7 Arkitektur

Applikationen följer i huvudsak SPA- (Single Page Application) och MVC-arkitekturen. Fördelen med SPA i detta fall är att vyerna uppdateras fort och behöver inte lagra data med hjälp av sessionStorage för att kunna dela resultat mellan sidor. Eftersom startsidan laddas enbart en gång kan data cachas i klasser istället. Detta förenklar koden och ökar prestanda. Stora objekt behöver inte konverteras från och till JSON-formatet och inte heller lagras och hämtas från webbläsarens minne, vilket kostar tid och lägger till extra kodrader till kodbasen. Nackdelen med SPA-arkitekturen är dock att den är svårare att optimera för sökmotorer, eftersom de inte har så mycket statiskt innehåll att genomsöka. Detta löses dock genom att ett statiskt innehåll med viktiga nyckelord läggs till index.html-filen.

MVC-arkitekturen används för att separera applikationens affärslogik från det grafiska gränssnittet. Detta gör kodbasen mer skalbar och testbar, eftersom de separata delar håller sig till sitt ansvarsområde. Affärslogiken kapslas in i modeller och UI-logiken i vyerna. Dessa är oberoende av varandra och kommunicerar enbart via App-klassen som fungerar här som en 'controller'. Både affärslogiken och UI-logiken kan därför utökas utan att koden för respektive del måste omskrivas. Eventuella fel är därutöver lättare att hitta med tanke på att varje modul testas is isolering. Affärslogiken är nämligen inte nästlad i UI-komponenterna.

## 4.8 Grafisk profil

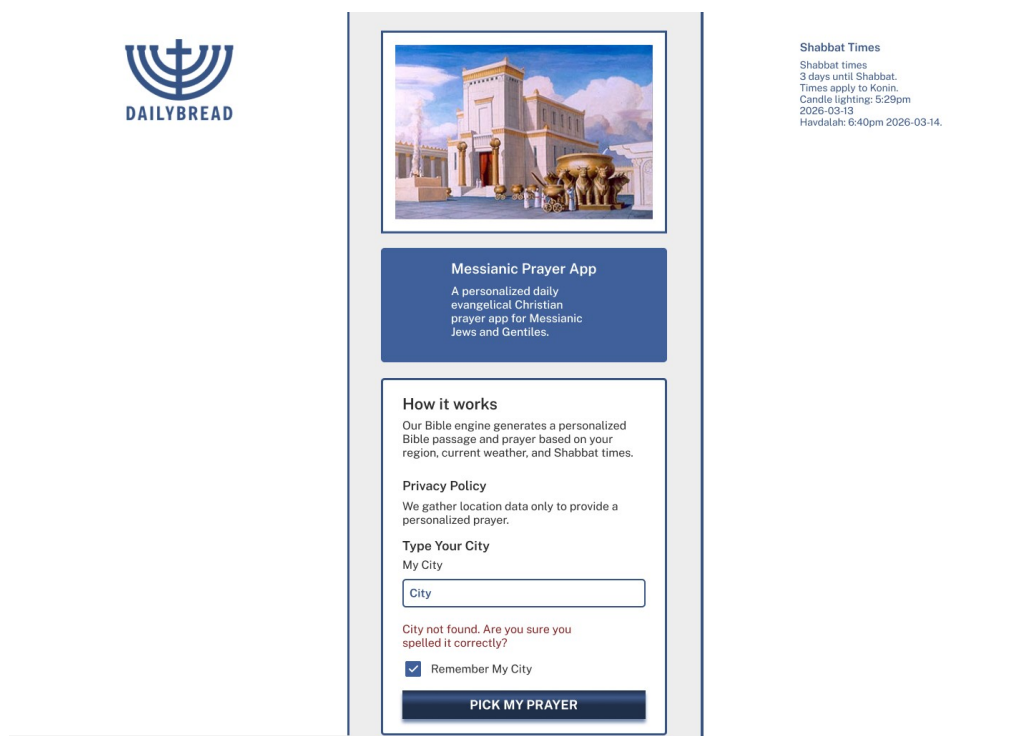
Eftersom applikationens huvudfunktionalitet kretsar kring läsning av bibelverser, väljs ett färgtema med god kontrast och ett 16-pixlars sans-seriftypsnitt för bästa läsbarheten. Vitt [44] och blått [45] nämns i Bibeln i anknytning till himmelsk estetik och blir därför DailyBread-appens primära färger. Färgerna har också en viktig symbolik inom psykologin; blått förknippas med professionalism och pålitlighet medan vitt med dygder [46]. De kompletterande färgerna blir ljusgrå och mörkgrå. För att lyfta fram applikationens meditativa syfte bygger Figma-prototypen på minimalism. Designen utgår från en minimalistisk enfärgad logotyp som är en kombination av den judiska sjuarmade ljusstaken 'menorah' [47] och det kristna korset. Menorah är en viktig religiös symbol inom både judendomen [47] och det evangelikala messianska samfundet [48]. Logotypen visas nedan i Figur 1.



Figur 1: DailyBread -Logotyp

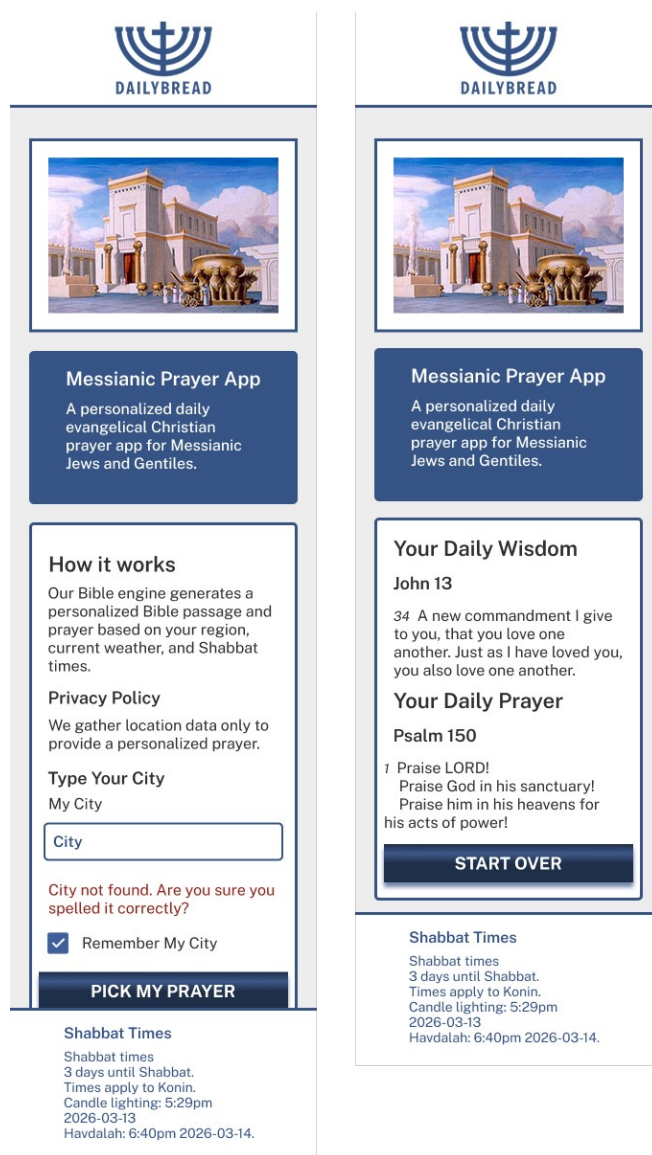
Två prototyp-versioner tas fram; en gemensam version för mobiler och surfplattor samt en version för datorer. Datorversionen har en okonventionell och innovativ layout som har inspirerats av ingången till det bibliska templet [49] och av templets innersta rum, det så kallade 'allra heligaste' [50]. Datorprototypen demonstreras i Figur 2 nedan.





**Figur 2: DailyBread – Version för dator**

Figur 2 visar även en liten bild av hur templet i Jerusalem kan ha sett ut enligt de beskrivningar som finns i Bibeln. De två vita ytorna på sidorna av layouten ska föra tankar till templets väggar, medan själva appen till det inre rummet, alltså till 'det allra heligaste'. Denna layoutstruktur är dock ett resultat av ett designproblem som uppstår under arbetsprocessen. I figur 3 illustreras designproblemet i den mobila versionen av prototypen.



Figur 3: DailyBread – Version för mobil och surfplatta

Figur 3 visar applikationens alla skärmar. Den minimalistiska designen, som bygger på endast två vyer som dynamiskt byts ut mot varandra, uppväcker en känsla av intimitet. Detta är ett medvetet designval som fungerar mycket väl på mobiler och smala surfplattor, men som är svårt att överföra till datorer. Det uppstår nämligen två designproblem i det här fallet. Om vyerna görs mer komplexa, riskerar applikationen att uppfattas som en opersonlig plattform snarare än en intim böneapp. Om vyerna däremot behåller sin minimalistiska form kommer datorversionen att uppfattas som tom och amatörmässig. Lösningen på detta problem är därför layouten i figur 2, som av en slump har visat sig passa väl ihop med applikationens koncept och tema.

Även valet av huvudbilden är en slump. Denna är ett substitut för en bild på Jesus som undervisar i den bibliska lagen. Bilden har emellertid många bruna färgtoner som står i konflikt med applikationens primära färger. Tempelbilden är den enda bild som hittas på internet som till fullo matchar DailyBread-appens färgtema och koncept och den inspirerar senare också layouten i datorversionen.

## 4.9 Automatisering och publicering

DailyBread-appen publiceras på GithubPages och processen automatiseras med hjälp av ett YAML-skript och GithubActions. Arbetsflödet körs varje gång projektets 'master'-gren uppdateras. Det installerar alla beroenden, bygger projektet med hjälp av byggverktyget Vite, sätter upp en korrekt bas-URL så att alla filer hittas, och laddar därefter upp den färdiga applikationen från mappen 'dist'. Mappen 'dist' är alltså den plats där Vite genererar den optimerade publiceringsklara versionen av projektet.

## 4.10 Testning

Testningen pågår dagligen parallellt med rapportskrivandet. Under tiden upptäcks flera fel relaterade till shabbaträknaren. Ursprungligen används DigiDate-API:t för att förenkla beräkningen av dagar kvar till shabbaten, men API:t visar sig vara opålitligt och ersätts av en intern funktionalitet som bygger på biblioteket 'date-fns'. En slumpmässig shabbatdag matas in 'New York' i platsfältet utan att shabbaträknaren uppdateras. Detta beror naturligtvis på tidszoner och åtgärdas med hjälp av ett litet hack. När shabbaträknaren returnerar '-1', visas istället ett anpassat meddelande som informerar att shabbaten pågår.

Under testningen byts dessutom Hebcal-endpointen ut mot en annan Hebcal-endpoint. Den sista går att modifiera så att judiska helgdagar ignoreras i respon- sen. En vecka före Påskan märks nämligen att shabbatdagen inte går att skilja från Pesach – den judiska versionen av Påskan [51]. Datumen för Pesach-dagarna blandas ihop med shabbatdagen och responsen går inte att filtrera så att bara datumet för shabbatdagen visas. Detta löses med hjälp av den nya endpointen och lämpliga parametrar.

## 5 Resultat

Projektarbetet resulterar i en välfungerande böneapp som uppfyller sitt mål. DailyBread-appen visar böner och visdomsord som är relevanta för användaren utifrån regionen, shabbatdagen och den dagliga väderprognosen. I det nuvarande tillståndet fungerar applikationen utan några fel både på mobiler och datorer och i följande webbläsare: FireFox, Edge, Chrome och Opera. Lighthouse-resultaten för prestanda, tillgänglighet, bästa praxis och sökmotoroptimering är 99, 100, 100 och 100 på både mobiler och datorer. Prestanda mäts dessutom med avstängd cache-lagring och via 3G-nätverk. Texten i applikationen uppfattas ha en god kontrast och läsbarhet, men på grund av brist på fler testanvändare går denna uppfattning inte att verifiera.

## 6 Slutsatser

DailyBread-appen är ingen komplex applikation. Den består bara av två vyer som byts ut mot varandra. Hela komplexiteten ligger i bibel-motorn som kombinerar data från flera API:er för att visa relevanta visdomsord och böner. Bibel-motorn är inte heller en riktig motor, utan en enkel mekanism som filtrerar data utifrån en uppsättning regler. Det som gör utvecklingen av projektet komplex är att projektets utvecklare själv ingår i målgruppen.

Å ena sidan är detta positivt. Testningen sker flera gånger om dagen och är utförlig. Funktionaliteten testas inte enbart ur ett tekniskt perspektiv som i fallet av tidigare projekt men det testas också ur ett användbarhetsperspektiv. Möter applikationen användarens behov? Känns appen personlig? Är användaren nöjd med de rekommenderade visdomsorden och bönerna?

Å andra sidan är detta negativt. Mycket av projektarbetet omfattar icke-tekniska uppgifter som bibelläsning och sökandet efter de bästa Psalm-avsnitten och mest passande Jesu ord. Det är svårt för utvecklaren att distansera sig från applikationen och släppa den. Ett skolprojekt är något som ska bockas av, men ett passionsprojekt är något som aldrig blir färdigt utan ständigt utvecklas.

## Källförteckning

- [1] UMJC, "Defining Messianic Judaism", <https://www.umjc.org/vision>, Hämtad 2026-03-19
- [2] IAMCS, "Find a Messianic Congregation", <https://www.iamcs.org/congregations/>, Hämtad 2026-03-19
- [3] Los Angeles Times, "In Europe's empty churches, prayer and confessions make way for drinking and dancing", <https://www.latimes.com/world-nation/story/2023-06-22/europe-empty-churches-repurposed> Publicerad 2023-06-22, Hämtad 2026-03-19
- [4] AP News, "Nordic people know how to beat the winter blues. Here's how to find light in the darkest months", <https://apnews.com/article/bewell-seasonal-affective-disorder-light-therapy-3a80cf76d63d80b4b7d0f84e7b67556a> Senast uppdaterad 2025-12-07, Hämtad 2026-03-19
- [5] BBC, "Why does sunshine makes you happy?" <https://www.bbc.com/weather/articles/c0qgxl2d5j3o> Publicerad 2025-08-08, Hämtad 2026-03-19
- [6] MDN Docs, "API", <https://developer.mozilla.org/en-US/docs/Glossary/API> Senast uppdaterad 2025-07-11, Hämtad 2026-03-20
- [7] AWS, "What is RESTful API?", <https://aws.amazon.com/what-is/restful-api/> , Hämtad 2026-03-20
- [8] MDN Docs "URIs", <https://developer.mozilla.org/en-US/docs/Web/URI>, Senast uppdaterad 2025-12-19, Hämtad 2026-03-20
- [9] MDN Docs, "Introducing asynchronous JavaScript", [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Extensions/Async\\_JS/Introducing](https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing) Senast uppdaterad 2026-02-11, Hämtad 2026-03-22
- [10] Oracle, Java Documentation, "Processes and threads", <https://docs.oracle.com/javase/tutorial/essential/concurrency/procthread.html> Hämtad 2026-03-22

- [11] MDN Docs, "Main thread",  
[https://developer.mozilla.org/en-US/docs/Glossary/Main\\_thread](https://developer.mozilla.org/en-US/docs/Glossary/Main_thread) Senast uppdaterad 2025-07-11, Hämtad 2026-03-22
- [12] MDN Docs, "Promise",  
[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Promise](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise), Senast uppdaterad 2025-01-21, Hämtad 2026-03-22
- [13] MDN Docs, "await",  
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await> Senast uppdaterad 2025-07-08, Hämtad 2026-03-22
- [14] MDN Docs, "Promise.all()",  
[https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Promise/all](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise/all) Senast uppdaterad 2025-08-19, Hämtad 2026-03-22
- [15] MDN Docs, "Ajax",  
<https://developer.mozilla.org/en-US/docs/Glossary/AJAX> Senast uppdaterad 2025-07-11, Hämtad 2026-03-23
- [16] MDN Docs, "Fetch API",  
[https://developer.mozilla.org/en-US/docs/Web/API/Fetch\\_API/Using\\_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) Senast uppdaterad 2025-08-20, Hämtad 2026-03-23
- [17] MDN Docs, "SPA (Single-page application)",  
<https://developer.mozilla.org/en-US/docs/Glossary/SPA>, Senast uppdaterad 2025-07-11, Hämtad 2026-03-23
- [18] Google Search Central, "Understand the JavaScript SEO basics",  
<https://developers.google.com/search/docs/crawling-indexing/javascript/javascript-seo-basics> Senast uppdaterad 2026-03-04, Hämtad 2026-03-23
- [19] MDN Docs, "MVC",  
<https://developer.mozilla.org/en-US/docs/Glossary/MVC> Senast uppdaterad 2025-07-11, Hämtad 2026-03-23
- [20] Geeks For Geeks "Benefit of using MVC",  
<https://www.geeksforgeeks.org/software-engineering/benefit-of-using-mvc/> Senast uppdaterad 2023-04-15, Hämtad 2026-03-23
- [21] MDN Docs, "CSS preprocessors", [https://developer.mozilla.org/en-US/docs/Glossary/CSS\\_preprocessor](https://developer.mozilla.org/en-US/docs/Glossary/CSS_preprocessor) Senast uppdaterad 2026-01-25, Hämtad 2026-03-23

- [22] Sass, "Documentation", <https://sass-lang.com/documentation/> Hämtad 2026-03-23
- [23] MDN Docs, "CSS animations",  
<https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Animations>  
Senast uppdaterad 2026-02-15, Hämtad 2026-03-23
- [24] MDN Docs, "@keyframes",  
<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@keyframes> Senast uppdaterad 2026-02-07, Hämtad 2026-03-23
- [25] Geeks For Geeks, "What is mashup in web technology?",  
<https://www.geeksforgeeks.org/websites-apps/what-is-a-mashup-in-web-technology/> Senast uppdaterad 2025-07-23, Hämtad 2026-03-23
- [26] Android Developers, "Quick Guides",  
<https://developer.android.com/develop/ui/compose/quick-guides> Senast uppdaterad 2025-02-06, Hämtad 2026-03-24
- [27] GeeksForGeeks, "Introduction to Java Swing",  
<https://www.geeksforgeeks.org/java/introduction-to-java-swing/> Senast uppdaterad 2025-07-23, Hämtad 2026-03-24
- [28] WxWidgets, "Hello World Example",  
[https://docs.wxwidgets.org/latest/overview\\_helloworld.html](https://docs.wxwidgets.org/latest/overview_helloworld.html) Hämtad 2026-03-24
- [29] Wikipedia, "Shabbat", <https://en.wikipedia.org/wiki/Shabbat> Senast uppdaterad 2026-03-13, Hämtad 2026-03-25
- [30] National Weather Service, "Heat Index Chart",  
<https://www.weather.gov/ffc/hichart> Hämtad 2026-03-25
- [31] Moustaq Karim Khan Rony och Hasnat M Alamgir , "High temperatures on mental health: Recognizing the association and the need for proactive strategies—A perspective",  
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10696165/> Publicerad 2023-12-04, Hämtad 2026-03-25
- [32] Scientific Reports, "Cold season air temperature as predictor of psychological well-being and mental health",  
<https://www.nature.com/articles/s41598-025-02486-x> Publicerad 2025-05-21, Hämtad 2026-03-25
- [33] National Snow and Ice Data Center, "Snow",  
<https://nsidc.org/learn/parts-cryosphere/snow/science-snow> Hämtad 2026-03-25



- [34] Harvard Health Publishing, "Shining a light on winter depression", [https://www.health.harvard.edu/newsletter\\_article/shining-a-light-on-winter-depression](https://www.health.harvard.edu/newsletter_article/shining-a-light-on-winter-depression) Publicerad 2024-09-30, Hämtad 2026-03-25
- [35] World Population Review, "Most Christian Countries 2026", <https://worldpopulationreview.com/country-rankings/most-christian-countries> Hämtad 2026-03-25
- [36] Jörg Stolz, Nan Dirk de Graaf, Conrad Hackett och Jean-Philippe Antonietti, "The three stages of religious decline around the world", <https://pmc.ncbi.nlm.nih.gov/articles/PMC12365078/> Publicerad 2025-08-19, Hämtad 2026-03-25
- [37] Global Christian Relief, "The 2026 GCR Red List", [https://drive.google.com/file/d/1DgIAYX3WznjnUAjMOMkvvx\\_HYG\\_o014ML/view](https://drive.google.com/file/d/1DgIAYX3WznjnUAjMOMkvvx_HYG_o014ML/view) Hämtad 2026-03-25
- [38] JSON, "Introducing JSON", <https://www.json.org/json-en.html> Hämtad 2026-03-27
- [39] Wikipedia, "Shabbat", <https://en.wikipedia.org/wiki/Shabbat> Senast uppdaterad 2026-03-13, Hämtad 2026-03-27
- [40] Geeks for Geeks, "What is automation?", <https://www.geeksforgeeks.org/software-engineering/what-is-automation/> Senast uppdaterad 2025-07-23, Hämtad 2026-03-30
- [41] Vite, "Getting started", <https://vite.dev/guide/> Hämtad 2026-03-30
- [42] Github Docs, "Understanding Github Actions", <https://docs.github.com/en/actions/get-started/understand-github-actions> Hämtad 2026-03-30
- [43] RedHat, "What is YAML?", <https://www.redhat.com/en/topics/automation/what-is-yaml> Publicerad 2023-03-03, Hämtad 2026-03-30
- [44] Andrew J. Elliot, "Color and psychological functioning: a review of theoretical and empirical work", <https://pmc.ncbi.nlm.nih.gov/articles/PMC4383146/> Publicerad 2015-04-02, Hämtad 2026-03-31
- [45] Biblestudy, "The Color White", <https://www.biblestudy.org/bible-study-by-topic/meaning-of-colors-in-the-bible/meaning-of-color-white.html>, Hämtad 2026-03-30

- [46] Biblestudy, "The Color Blue", <https://www.biblestudy.org/bible-study-by-topic/meaning-of-colors-in-the-bible/meaning-of-color-blue.html>  
Hämtad 2026-03-30
- [47] Wikipedia, "Menorah", [https://en.wikipedia.org/wiki/Temple\\_menorah](https://en.wikipedia.org/wiki/Temple_menorah)  
Senast uppdaterad 2026-03-26, Hämtad 2026-04-01
- [48] Wikipedia "Messianic Judaism",  
[https://en.wikipedia.org/wiki/Messianic\\_Judaism](https://en.wikipedia.org/wiki/Messianic_Judaism) Senast uppdaterad  
2026-03-26, Hämtad 2026-04-01
- [49] Wikipedia, "Temple in Jerusalem",  
[https://en.wikipedia.org/wiki/Temple\\_in\\_Jerusalem](https://en.wikipedia.org/wiki/Temple_in_Jerusalem) Senast uppdaterad  
2026-03-21, Hämtad 2026-04-01
- [50] Wikipedia, "Holy of Holiest",  
[https://en.wikipedia.org/wiki/Holy\\_of\\_Holies](https://en.wikipedia.org/wiki/Holy_of_Holies) Senast uppdaterad 2026-  
03-20, Hämtad 2026-04-01
- [51] MyJewishLearning, "Easter and Passover",  
<https://www.myjewishlearning.com/article/easter-and-passover/> Hämtad  
2026-04-02